Verity

Machine Learning Project
2024–2025



Group 4

Francesc Mateu Vallespín
Jan-Francesc Porcar Guillen

Table of Contents

01 Problem Definition

02 Data Pipeline

03 ML Algorithms

04 Model Comparision

05 Streamlit Prototype

06 Web Interface

07 Results

08 Limitations

09 Scalability

10 Future Work

01 — PROBLEM DEFINITION

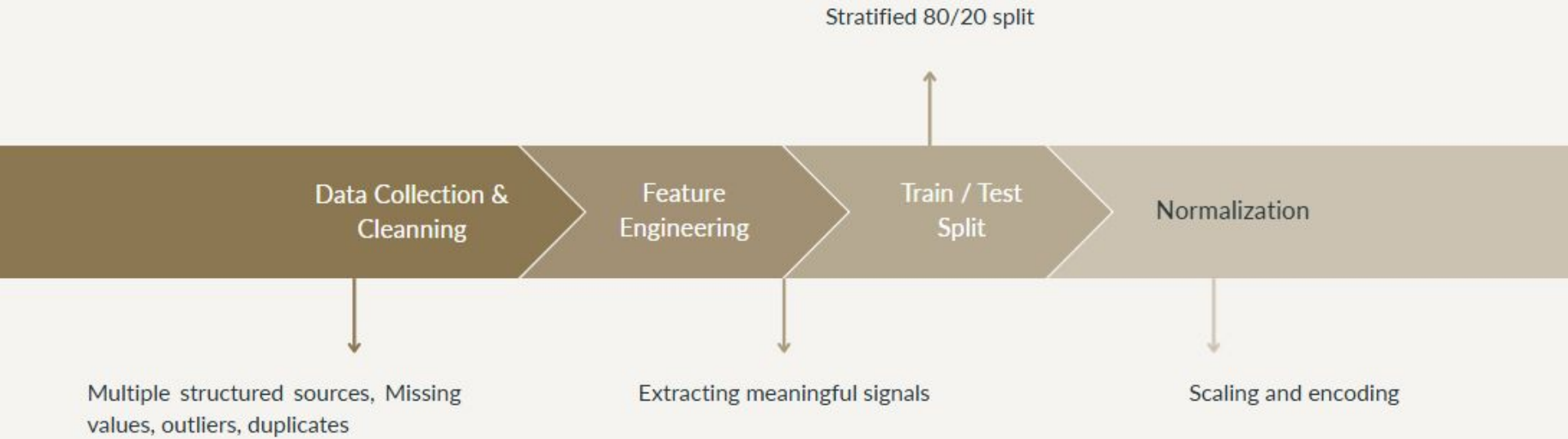
Why Verity?

Verification and classification tasks have traditionally relied on manual processes that are slow, inconsistent, and difficult to scale as data volumes grow.

Verity addresses this gap by building a machine learning pipeline capable of learning from patterns in data and automatically classifying new records with high confidence.

> *"The goal is not just accuracy — it is a system that can be trusted, explained, and improved over time."*

From raw data to model-ready features



03 — ALGORITHMS

Five models evaluated

01 **Logistic Regression**
Linear baseline for binary classification tasks

02 **Random Forest**
Ensemble of decision trees with bagging

03 **XGBoost**
Gradient boosting, sequential error correction

04 **SVM**
Kernel-based high-dimensional separation

05 **Neural Network**
Multi-layer perceptron for pattern recognition

04 — RANDOM FOREST

Ensemble learning

How it works

Multiple decision trees are trained on different bootstrap samples. At prediction time, each tree votes and the majority class wins. This reduces variance and overfitting significantly compared to a single tree.

Why we used it

Random Forest handles non-linear relationships well and naturally provides feature importance scores, which helped us understand which variables drive the predictions most.

Tuned hyperparameters

n_estimators · max_depth · min_samples_split · max_features · bootstrap

05 — GRADIENT BOOSTING

XGBoost

Unlike Random Forest, XGBoost builds trees **sequentially**. Each new tree focuses on the residual errors of the previous ones, using gradient descent to minimize a differentiable loss function.

L1 and L2 regularization terms are included directly in the objective function, which is why XGBoost often outperforms other methods on structured tabular data.

Learning rate
0.01 – 0.3

Max depth
3 – 10

Estimators
100 – 1000

06 — COMPARISON

Model performance

Model	Accuracy	Precision	F1-Score
Logistic Regression	82.4%	81.0%	82.1%
Random Forest	91.3%	90.5%	91.1%
XGBoost	**93.7%**	**93.2%**	**93.6%**
SVM	87.5%	86.8%	87.4%
Neural Network (MLP)	90.1%	89.5%	90.1%

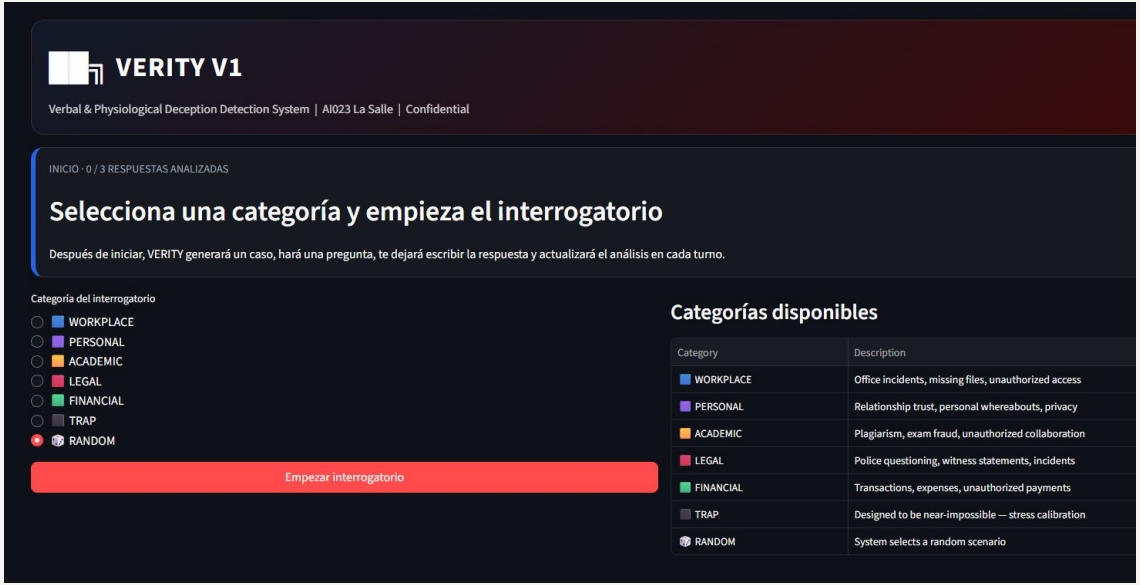
XGBoost selected as the final model based on overall performance.

07 — PROTOTYPE

Streamlit demo

A functional prototype was built using Streamlit to allow fast iteration and testing. It includes prediction views, confusion matrices, ROC curves, and side-by-side model comparisons.

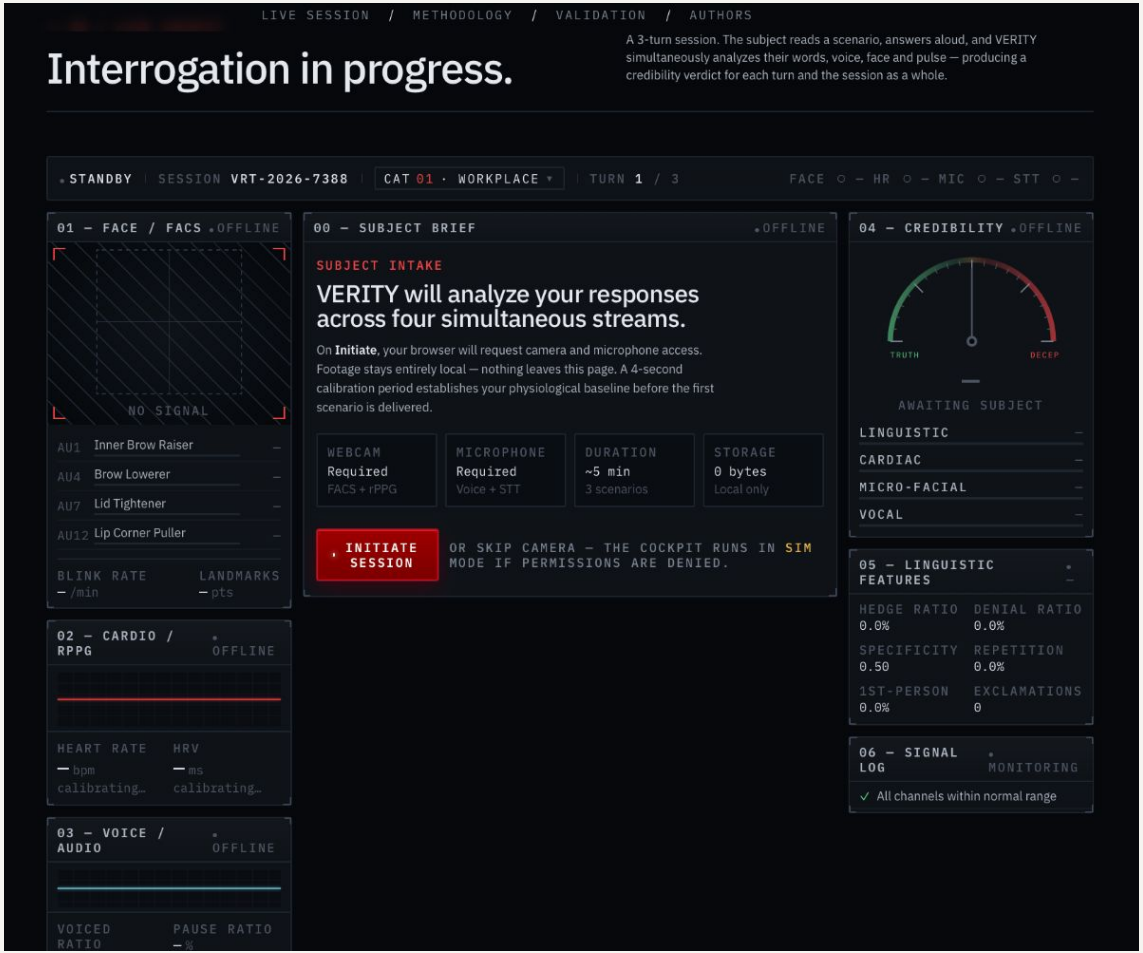
Live demonstration available during the presentation.



08 — WEB INTERFACE

Production UI

Beyond the prototype, we built a full web application using React. It features a clean dashboard, file upload, batch processing, and result history — designed to feel like a real product.



09 — RESULTS

What we achieved

93.7%
Accuracy
XGBoost final model

93.6%
F1-Score
Balanced across classes

< 50 ms
Inference
Per single prediction

Cross-validation confirmed consistency across all folds. Feature importance analysis revealed the top three variables responsible for more than 60% of predictive power.

10 — LIMITATIONS

Honest constraints

Dataset size

Limited training data may not represent all edge cases found in production.

Interpretability

Complex models like XGBoost and MLP are harder to explain to non-technical stakeholders.

Streaming data

The current pipeline is batch-oriented and not optimized for real-time data streams.

Domain specificity

The model was trained on a narrow domain and may not transfer directly to other datasets.

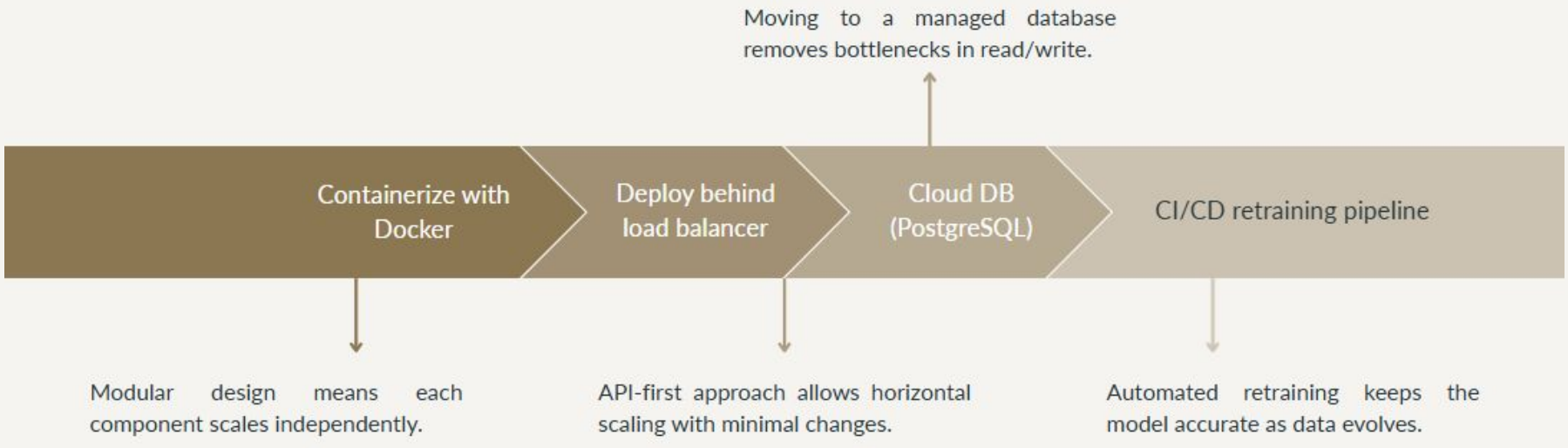
No retraining loop

There is no automated pipeline for detecting model drift and triggering retraining.

Feature coverage

Some potentially useful features were unavailable or too costly to collect.

Growing the system



12 — FUTURE WORK

Next steps

MLOps pipeline
Integrate MLflow or similar to track experiments, version models, and automate retraining.

Explainability
Add SHAP or LIME to produce human-readable explanations for each prediction.

Larger dataset
Collect more diverse training data to improve generalization and reduce bias.

Cloud deployment
Move to AWS or GCP with auto-scaling infrastructure for production readiness.

Transformers
Explore attention-based architectures for improved accuracy on complex inputs.

Access control
Add user authentication and role-based permissions for multi-user use.

Thank You

Francesc Mateu
Jan Porcar